LAB
12

Operating Systems · FCIM / FAF · UTM · 2026-2027

Producer, Consumer and the Other Tools

A Track A: the real OS, from the outside (Linux) · 90 minutes

AT A GLANCE

Part 1	Producer and consumer	~30 min
Part 2	The condition variable	~20 min
Part 3	Readers and writers	~20 min
Part 4	Build a deadlock, then remove it	~20 min

Goal

In Lab 11 you used one semaphore at a time: as a lock, as a signal, as a counter. Today the tools work together. You solve the two classic problems of every operating systems course, producer and consumer, readers and writers, you meet the condition variable, and you build (and then break) a deadlock.

Before you start

- Same laptop, same tools as Lab 11, nothing to install. New folder: `mkdir ~/os-lab12 && cd ~/os-lab12`.
- Every program is complete: type it, compile it with -pthread, run it. Answers go in report.md.

Minimum to pass

Parts 1 and 2 run, with their Observe answers. Parts 3 and 4 bring the rest of the grade.

PART 1 Producer and consumer ~30 min

A producer puts items into a buffer of 4 slots, a slow consumer takes them out. Three tools work together: empty counts the free slots (starts at 4), full counts the filled slots (starts at 0), and a mutex protects the buffer itself. Every pipe in Unix works exactly like this, inside the kernel.

FILE prodcons.c

```
/* prodcons.c - producer and consumer around a buffer of 4 slots.
 *   empty = free slots (starts at SIZE), full = filled slots (starts at 0), lock = the
buffer itself.
 * Build:  gcc -Wall -pthread -o prodcons prodcons.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;     /* count only for printing */
```

```
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);               /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put  %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);                /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);                /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("            take %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);               /* one more free slot */
        usleep(300000);                 /* the consumer is slow */
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ ./prodcons
```

Expected output:

EXPECTED OUTPUT

```
put  1   (in buffer: 1)
put  2   (in buffer: 2)
put  3   (in buffer: 3)
put  4   (in buffer: 4)
            take 1   (in buffer: 3)
put  5   (in buffer: 4)
            take 2   (in buffer: 3)
```

```
put  6   (in buffer: 4)
            take 3   (in buffer: 3)
put  7   (in buffer: 4)
            take 4   (in buffer: 3)
put  8   (in buffer: 4)
            take 5   (in buffer: 3)
            take 6   (in buffer: 2)
            take 7   (in buffer: 1)
            take 8   (in buffer: 0)
```

Observe (write in report.md)

1. The producer is fast, yet "in buffer" never goes above 4. Which call stops it?
2. Who waits at the start: the consumer or the producer? Which call makes it wait?
3. In producer(), swap the first two lines: lock the mutex BEFORE sem_wait(&empty). Run it. What happens when the buffer is full, and why? (Ctrl-C to stop it, then put the lines back.)

PART 2 The condition variable ~20 min

The problem of order.c from Lab 11 (B must read only after A has written), solved another way. The state is a normal variable, ready, protected by a mutex. The condition variable is only the place where B sleeps until A says "something changed". pthread_cond_wait releases the mutex, sleeps, and takes the mutex again before it returns.

FILE condvar.c

```
/* condvar.c - the order.c problem, solved with a mutex and a condition variable.
 * Build:  gcc -Wall -pthread -o condvar condvar.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;                                  /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);              /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                            /* while, not if: re-check after waking
*/
        printf("B: not ready, I sleep\n");
```

```
        pthread_cond_wait(&changed, &lock);     /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
```

Expected output:

EXPECTED OUTPUT

```
B: not ready, I sleep
A: message written
B: I read "hello"
```

Observe (write in report.md)

1. In order.c (Lab 11) the semaphore itself remembered that the message was ready. Here, which variable remembers it?
2. Replace "while (!ready)" with "if (!ready)", compile, run. Does it still work here?
3. Then why do we write while? Give one reason why a thread can wake up and still find ready == 0.

PART 3 Readers and writers ~20 min

Many readers may read together; a writer must be alone. Two semaphores: room (1 = free for a writer) and rlock (a lock around the counter readers). The first reader in locks the room, the last one out unlocks it.

FILE readers.c

```
/* readers.c - readers and writers with two semaphores.
 *   Many readers may read together; a writer must be alone.
 * Build:  gcc -Wall -pthread -o readers readers.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;                  /* the shared data */
int readers = 0;                /* how many are reading now */
sem_t rlock;                    /* protects the counter readers */
```

```
sem_t room;                     /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)         /* the first reader in locks out the writers */
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d   (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)         /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);            /* alone in the room */
    value += 10;
    printf("writer %ld writes %d  (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
```

Expected output:

EXPECTED OUTPUT

```
writer 1 writes 10  (alone)
reader 1 reads 10   (readers inside: 1)
reader 2 reads 10   (readers inside: 2)
reader 3 reads 10   (readers inside: 3)
writer 2 writes 20  (alone)
```

Observe (write in report.md)

1. Three readers are inside at the same time, but a writer is always alone. Which two lines of reader() make that possible?
2. Why does the counter readers need its own lock (rlock)? What could go wrong without it?
3. Imagine readers that keep arriving without a pause. What happens to writer 2? (This problem has a name: starvation.)

PART 4 Build a deadlock, then remove it ~20 min

Two threads, two mutexes, taken in opposite order. Each one holds what the other needs, and both wait forever. Run it with timeout, so your terminal comes back after 5 seconds (exit code 124 = timeout had to kill it):

FILE deadlock.c

```
/* deadlock.c - two threads, two mutexes, taken in opposite order.
 * Build:  gcc -Wall -pthread -o deadlock deadlock.c
 * Run:    timeout 5 ./deadlock; echo "exit code: $?"      (124 = killed by timeout)
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);             /* waits for two */
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&B);             /* the fix: lock A first here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&A);             /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
```

```
    printf("done\n");
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock; echo "exit code: $?"
```

On your screen you see the two lines "I have A, I want B" and "I have B, I want A", then nothing for 5 seconds, then:

TERMINAL

```
exit code: 124
```

Now the fix: in two(), lock A first, then B, the same order as one() (swap the two pthread_mutex_lock lines and the two unlock lines). Compile and run again:

Expected output:

EXPECTED OUTPUT

```
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
```

Observe (write in report.md)

1. In the stuck run, which thread holds which mutex, and which one is each thread waiting for? Draw the circle.
2. Remove both sleep(1) lines from the ORIGINAL version and run it ten times. Does it still always hang? Why is that worse, not better?
3. In one sentence: why can taking the locks in the same order never deadlock?

What you hand in

- prodcons.c, condvar.c, readers.c, deadlock.c (the fixed version)
- report.md with the Observe answers of Parts 1 to 4.

Submit by git: push to your course repository; the assistant pulls it.

Criterion	Points
Part 1: producer/consumer runs, the three questions answered	**3 p**
Part 2: condvar runs, Observe answered	**3 p**
Part 3: readers and writers, Observe answered	**2 p**
Part 4: deadlock seen and fixed, Observe answered	**2 p**
Total	**10**